Clustering

Aprendizaje no supervisado · Semana 4

Antonio Miguel López Rodríguez
Sistemas Industriales Inteligentes · MAIIND
Curso 2026–2027

Aprendizaje no supervisado

Supervisado

Cada ejemplo trae la respuesta: el valor que hay que predecir o la clase.

El modelo aprende a reproducirla. Regresión y clasificación, semanas 2 y 3.

No supervisado

Solo hay variables de entrada, sin respuesta.

El algoritmo busca estructura en los datos: grupos, direcciones en las que varían, puntos raros.

En planta: reconocer formas de operar, separar regímenes o resumir muchos sensores en pocas variables.

¿Qué significa similar?









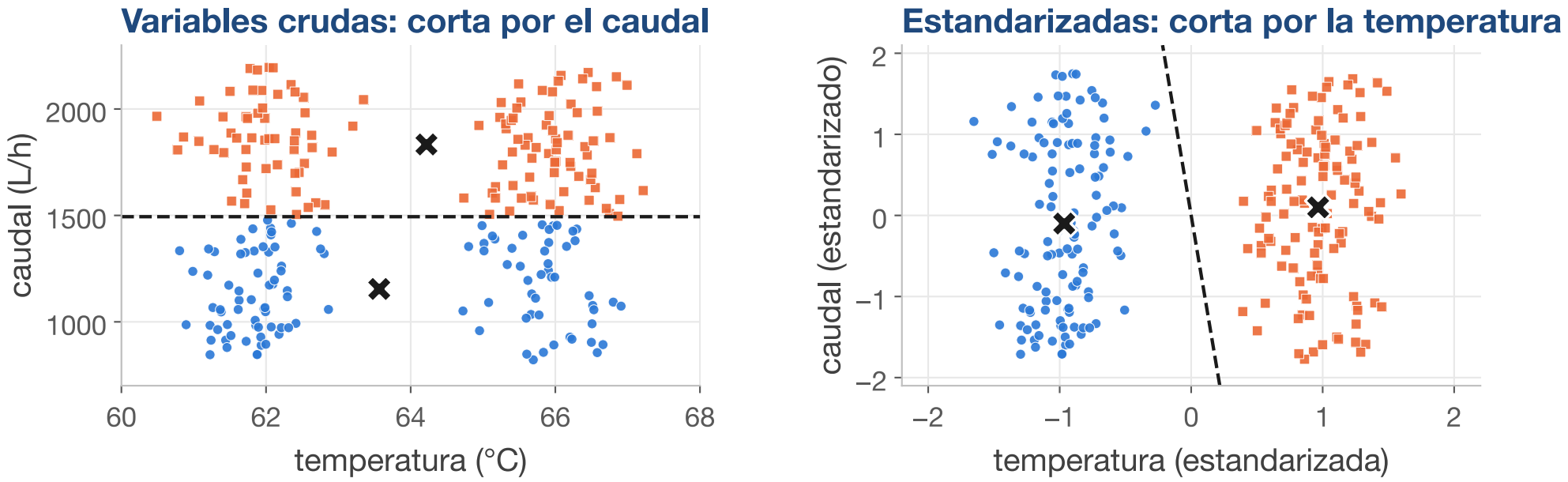
Qué es un buen agrupamiento

Agrupar (*clustering*) es repartir los puntos en grupos sin que nadie diga cuáles son.

- Los puntos de un mismo grupo se parecen entre sí: distancias **dentro** del grupo pequeñas.
- Los de grupos distintos se diferencian: distancias **entre** grupos grandes.
- «Parecerse» hay que definirlo. Lo habitual es la distancia euclídea entre filas de variables numéricas.

No hay una respuesta correcta con la que comparar: un agrupamiento se juzga por su forma y por su utilidad.

La escala decide qué está cerca



Sin estandarizar, 4 °C entre las nubes pesan como 4 L/h, nada al lado de los cientos de L/h del caudal. Con `StandardScaler` (media 0, desviación típica 1), las dos variables pesan igual.

Familias de métodos

Familia	Idea	Qué hay que fijar
Partición: **k-means**	Cada grupo tiene un centro; cada punto va al más cercano	El número de grupos k
Jerárquico	Fusiona grupos paso a paso y forma un árbol	Dónde cortar el árbol
Densidad: **DBSCAN**	Un grupo es una zona densa; lo aislado es ruido	Radio y número mínimo de vecinos
Probabilístico: **mezcla de gaussianas**	Cada grupo es una gaussiana; cada punto tiene una probabilidad de pertenecer a cada una	El número de componentes

En clase usamos k-means. Los demás aparecen al final, para cuando k-means no basta.

k-means

Un centro por grupo y cada punto, al centro más cercano

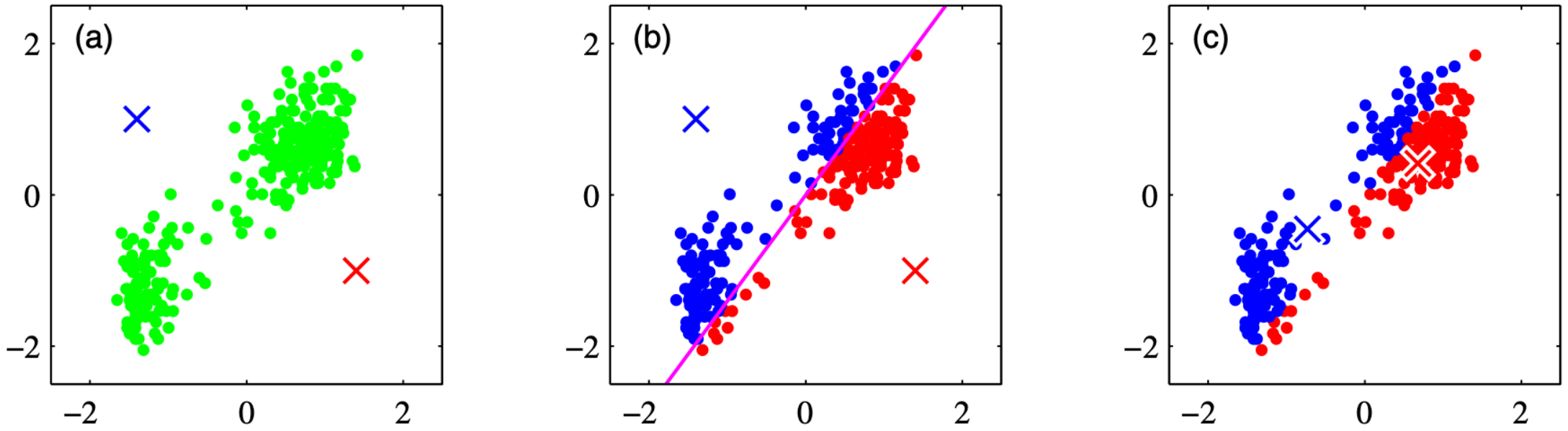
El algoritmo

1. Elegir el número de grupos **k** y colocar k centros iniciales.
2. **Asignar** cada punto al centro más cercano.
3. **Mover** cada centro a la media de los puntos que tiene asignados.
4. Repetir 2 y 3 hasta que los centros dejen de moverse.

---

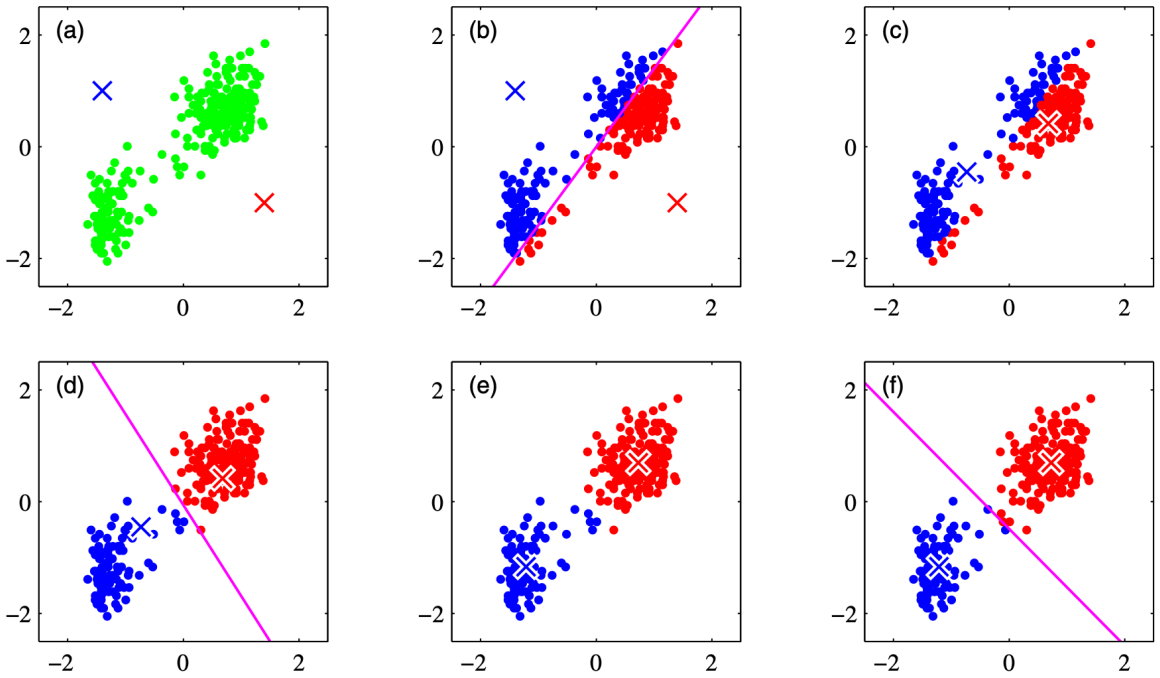
k lo decidimos nosotros. k-means no sabe cuántos grupos hay: busca exactamente los que se le piden.

Ejemplo de k-means



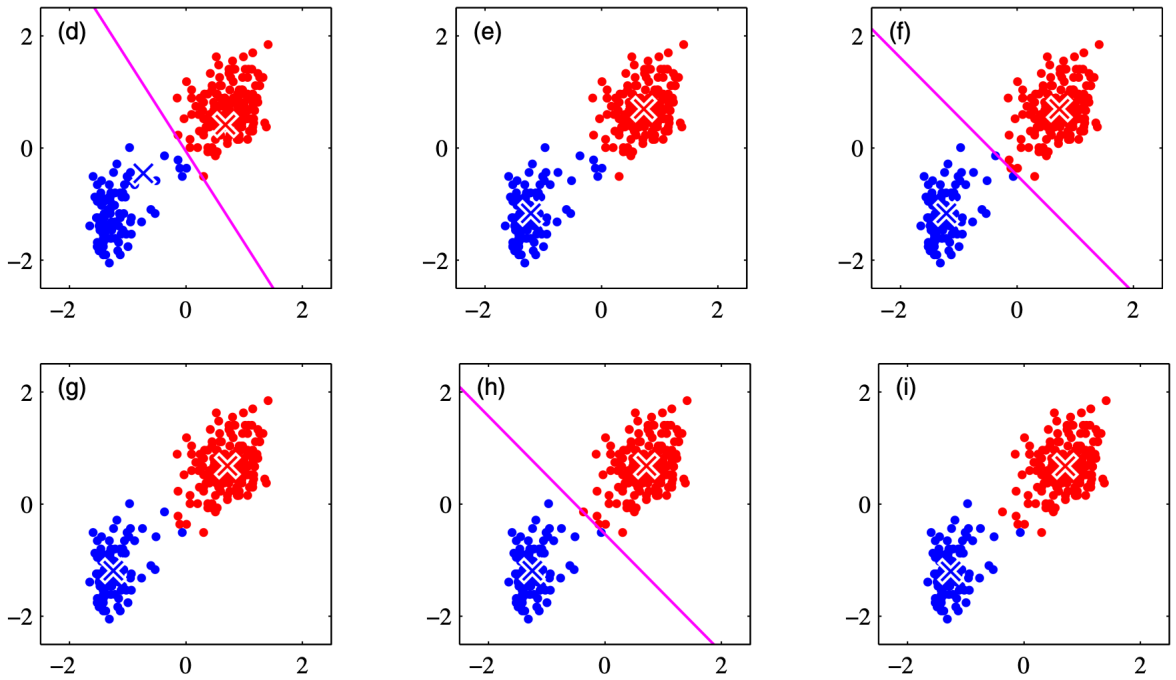
Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, fig. 9.1 (datos Old Faithful reescalados).

Ejemplo de k-means



Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, fig. 9.1.

Ejemplo de k-means

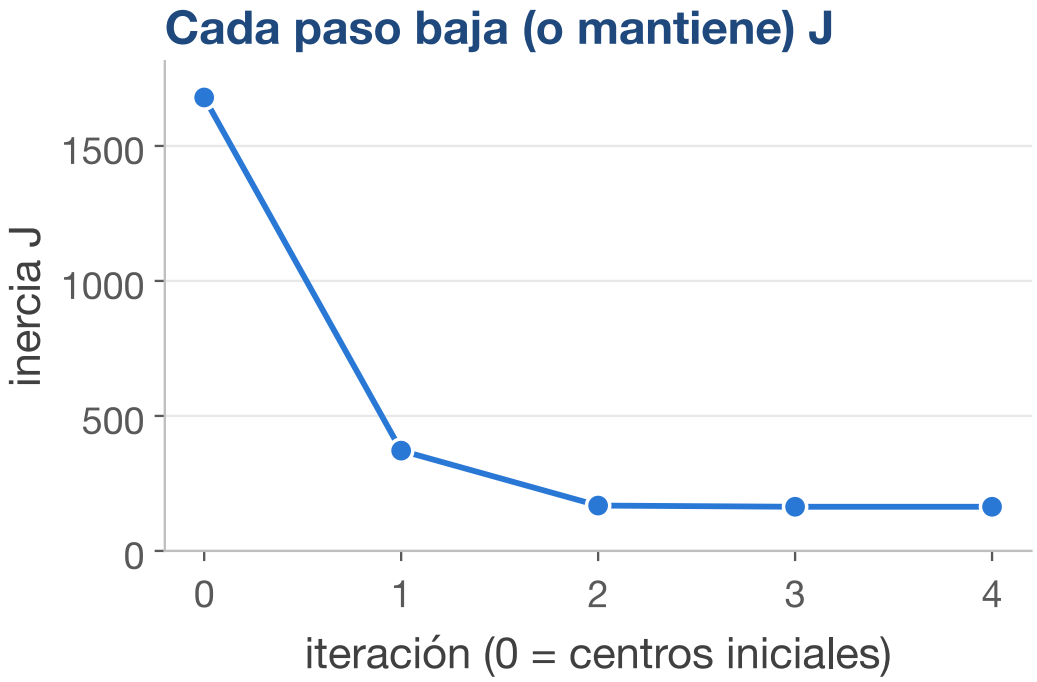


Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer, fig. 9.1.

Lo que minimiza: la inercia

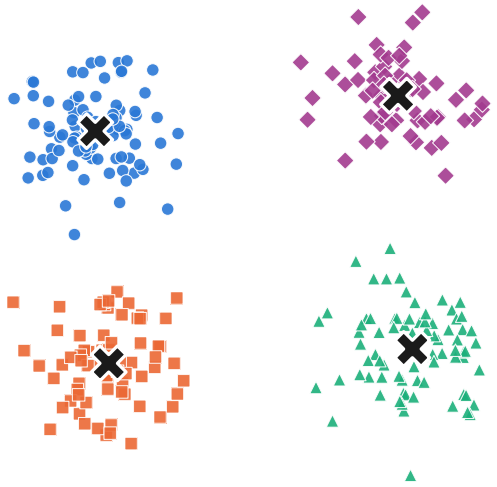
$$J = \sum_{i=1}^{n} \| x_i - \mu_{c(i)} \|^2$$

- x_i: punto i; $\mu_{c(i)}$: centro del grupo al que está asignado.
- Suma de las distancias al cuadrado de cada punto a su centro. En scikit-learn, `inertia_`.
- Asignar y mover nunca la aumentan: el algoritmo siempre termina.

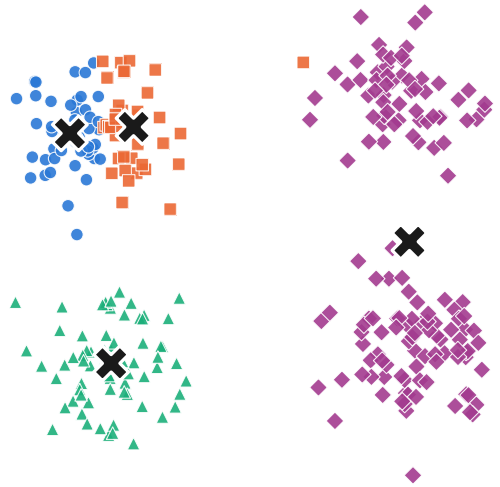


Depende del arranque: mínimos locales

Arranque A: inercia = 282



Arranque B: inercia = 978



`KMeans(n_init=10)` repite el algoritmo con 10 arranques y se queda con el de menor inercia. `random_state` fija el azar para que el resultado se pueda repetir.

k-means en Python

```
escalador = StandardScaler()
X = escalador.fit_transform(datos[variables])

modelo_kmeans = KMeans(n_clusters=4, n_init=10,
                       random_state=0)
grupo = modelo_kmeans.fit_predict(X)

centros = modelo_kmeans.cluster_centers_
inercia = modelo_kmeans.inertia_
```

`fit_predict` ajusta el modelo y devuelve el grupo de cada fila: 0, 1, …, k–1.

`cluster_centers_`: un centro por grupo, en unidades estandarizadas.

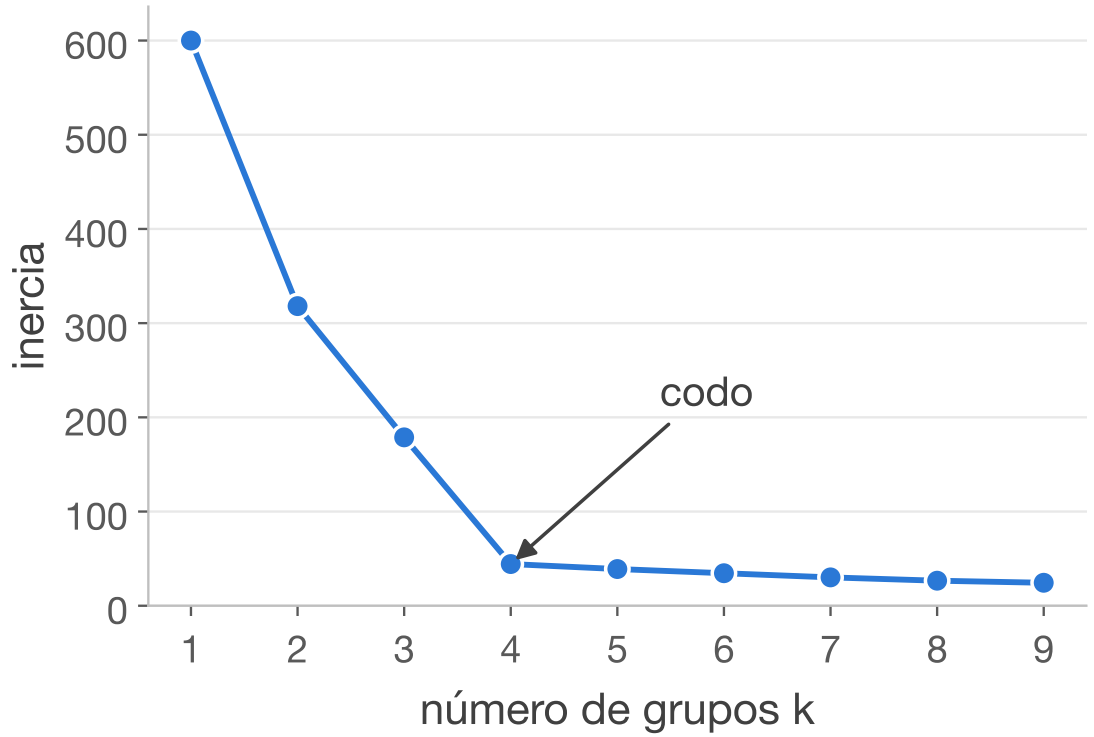
`escalador.inverse_transform(centros)` los devuelve a las unidades originales.

Importaciones: `StandardScaler` de `sklearn.preprocessing` y `KMeans` de `sklearn.cluster`.

¿Cuántos grupos?

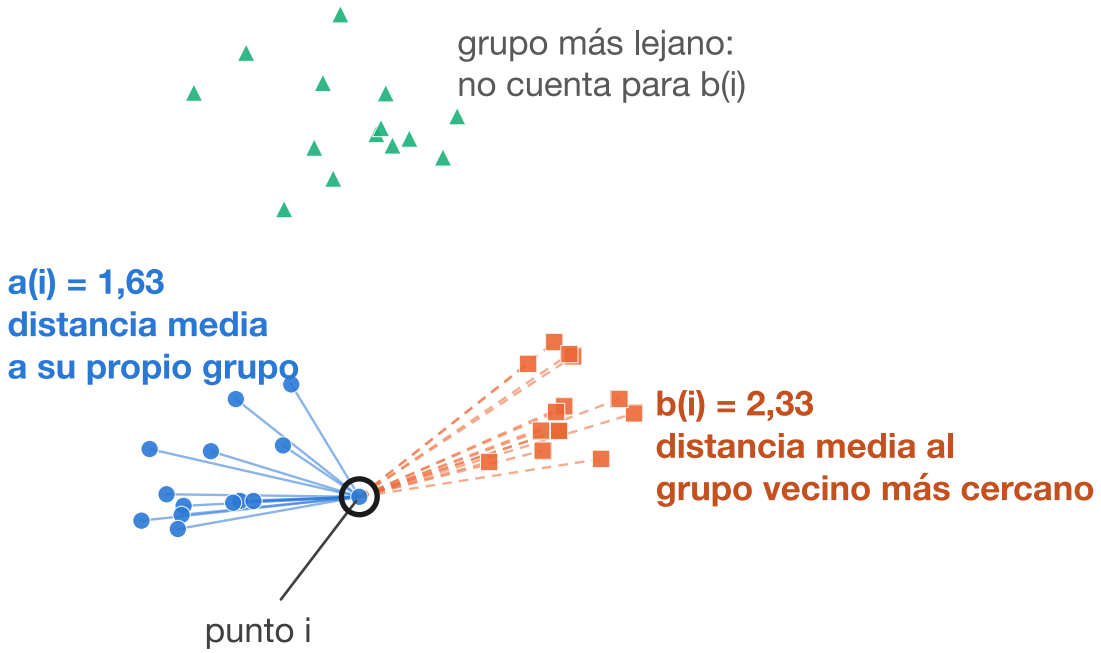
El codo, la silueta y el gráfico de silueta

El método del codo



- La inercia **siempre baja** al aumentar k: con un grupo por punto valdría 0.
- No se busca el mínimo, sino el **codo**: el k a partir del cual añadir grupos apenas la reduce.
- Con datos reales el codo suele ser difuso. Hace falta otro criterio.

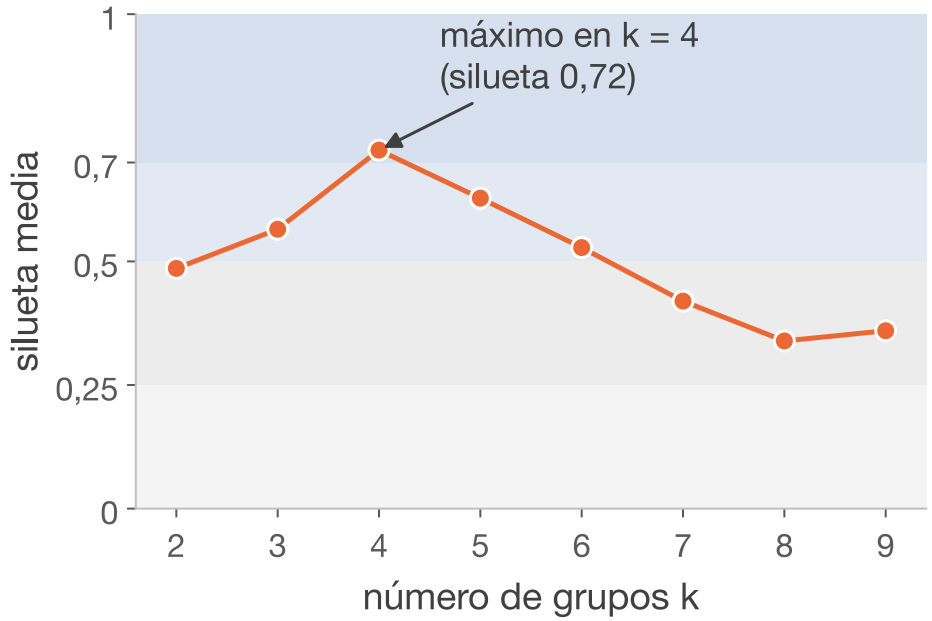
La silueta de un punto



$$s(i) = \frac{b(i) - a(i)}{\max\{a(i),\, b(i)\}}$$

- **a(i)**: distancia media a los demás puntos de su grupo.
- **b(i)**: distancia media a los puntos del grupo vecino más cercano.
- Va de –1 a 1. Cerca de 1, bien dentro de su grupo; cerca de 0, en la frontera; **negativa**, más cerca del grupo vecino que del suyo.

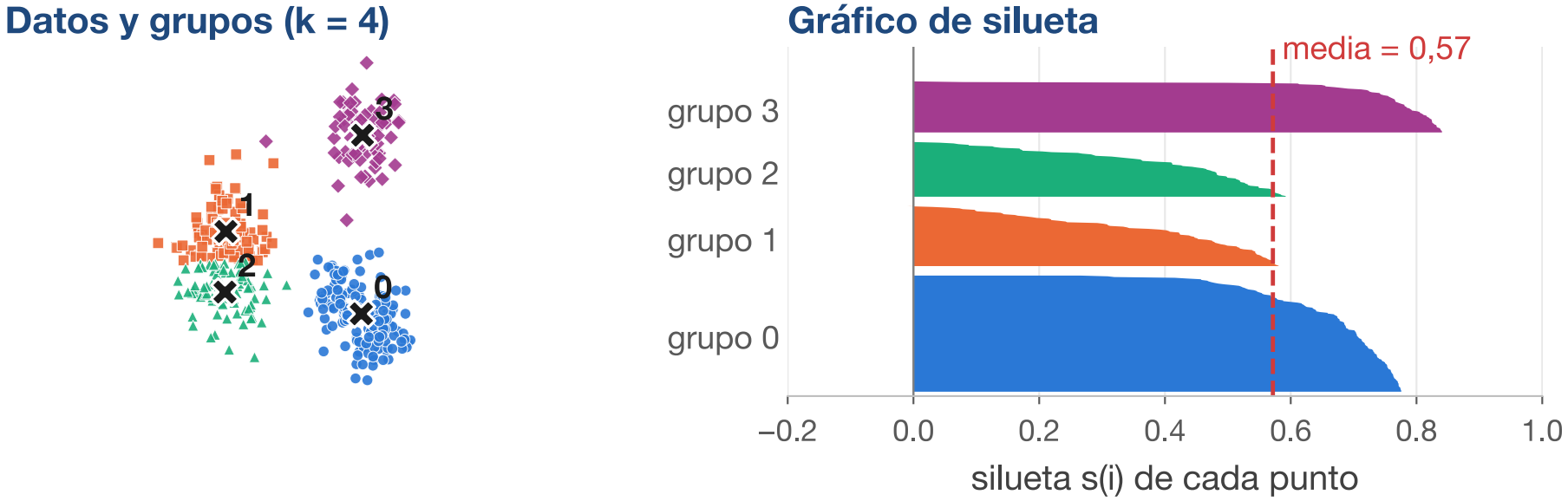
La silueta media para elegir k



- La media de s(i) en todos los puntos resume el agrupamiento: `silhouette_score(X, grupo)`.
- Se elige el k con la **silueta media más alta**.
- Referencia orientativa: menos de 0,25, sin estructura clara; 0,25–0,5, débil; 0,5–0,7, razonable; más de 0,7, fuerte.

Codo y silueta no siempre coinciden. Si discrepan, hay que justificar el k elegido.

El gráfico de silueta



Una barra por punto, agrupadas por grupo y ordenadas de mayor a menor. La línea roja es la silueta media. Se calcula con `silhouette_samples(X, grupo)`.

Qué mirar en el gráfico de silueta

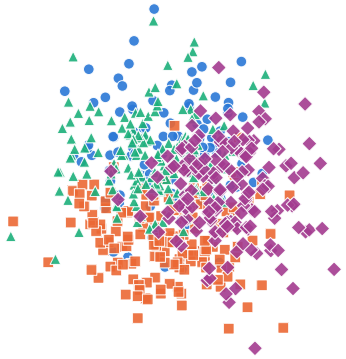
- **Grupos enteros bajo la media**: grupos mal definidos. Aquí, el 1 y el 2, que se solapan.
- **Grosor**: cuántos puntos tiene cada grupo.
- **Barras negativas**: puntos más cerca de otro grupo que del suyo.

Grupo	Puntos	Silueta media
0	160	0,67
1	83	0,39
2	76	0,40
3	71	0,75
Todos	390	0,57

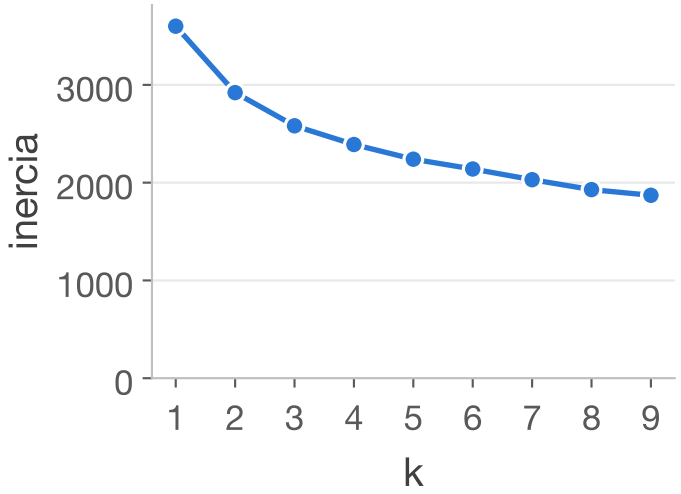
La media dice «razonable», pero esconde dos grupos mal separados. Mira siempre el gráfico.

Si no hay grupos, k-means los inventa

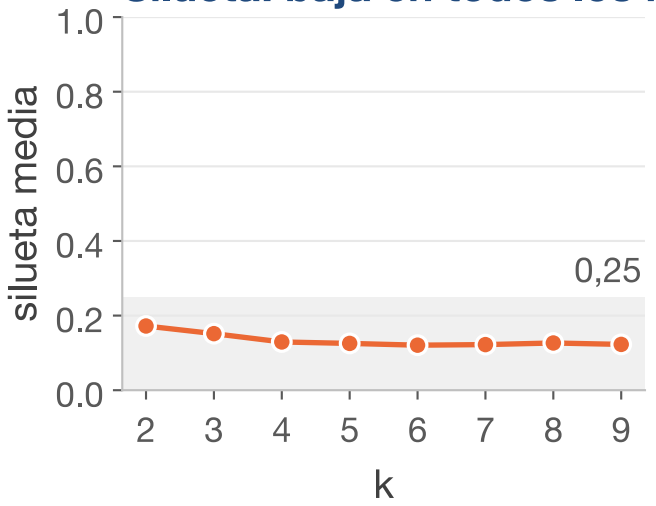
k = 4: dos de las 6 variables



Codo: no hay ninguno claro



Silueta: baja en todos los k



k-means siempre devuelve k grupos. Una silueta baja para todos los k indica un continuo, no grupos separados: los grupos pueden servir para resumir, pero no son tipos naturales.

Después de agrupar (1/3): ¿salen los mismos grupos?

Punto	A	B	C	D	E	F	G	H
Grupo con la semilla 0	0	0	0	1	1	1	2	2
Semilla 1, caso 1	2	2	2	0	0	0	1	1
Semilla 1, caso 2	2	2	**0**	0	0	**1**	1	1

Caso 1: mismos grupos, otro número

	0	1	2
0	0	0	**3**
1	**3**	0	0
2	0	**2**	0

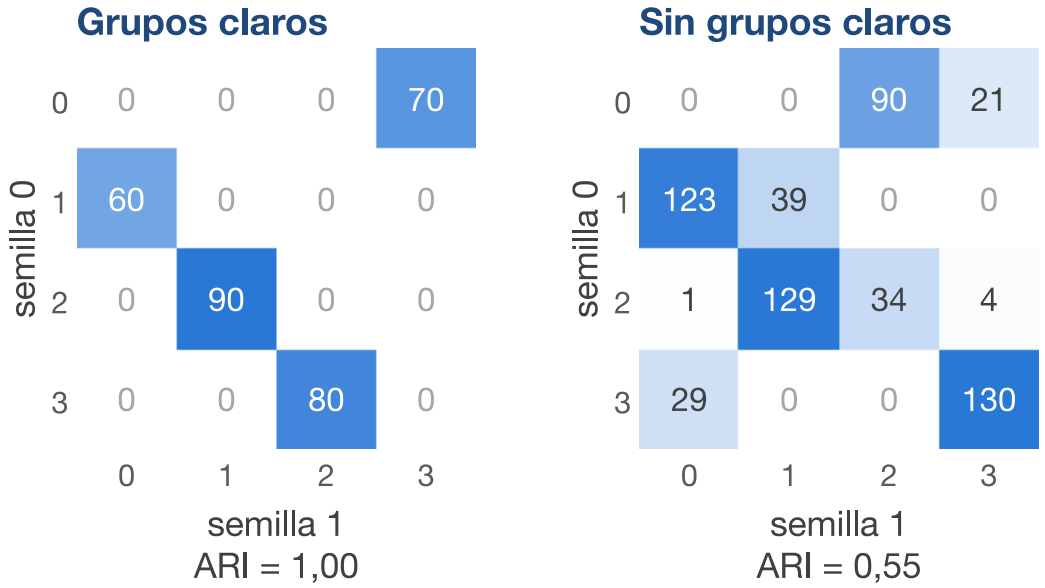
Caso 2: C y F cambian de grupo

	0	1	2
0	1	0	**2**
1	**2**	1	0
2	0	**2**	0

`pd.crosstab(semilla_0, semilla_1)` cuenta los puntos de cada pareja (filas: semilla 0; columnas: semilla 1). Mismos grupos: **una casilla llena por fila**.

Después de agrupar (1/3): con grupos claros y sin ellos

- k-means arranca al azar. **Con grupos claros**, cualquier semilla llega al mismo reparto: una casilla por fila.
- **Sin grupos claros**, cada semilla reparte distinto: las filas se reparten entre varias casillas.
- `adjusted_rand_score` resume la tabla: **1**, mismos grupos; cerca de **0**, como al azar.



Izquierda: los cuatro grupos claros del codo. Derecha: los datos sin grupos de la diapositiva 21.

Después de agrupar (2/3): centros en unidades reales

```
escalador = StandardScaler()
X = escalador.fit_transform(datos)
# el escalador guarda mean_ y scale_
# ... k-means con X ...
centros_z = modelo_kmeans.cluster_centers_
centros = escalador.inverse_transform(centros_z)
```

`inverse_transform` calcula `centros_z × scale_ + mean_` con los valores que guardó el escalador al ajustarse.

Ejemplo	°C	L/h
`mean_`	63,9	1513,7
`scale_`	2,07	390,3
centro 0 (z)	−0,97	−0,10
centro 0 real	61,9	1475

Hay que usar el mismo escalador. Sale la media de cada grupo, como `groupby(grupo).mean()`.

Después de agrupar (3/3): perfil con otras variables

Con qué variables

Con las que **no entraron** en el agrupamiento: una medida de calidad, la fecha, el turno, el lote.

Si entraran, los grupos se formarían con ellas y la conclusión sería circular.

Qué se mira

Para cada grupo, su tamaño y la media de esas variables, frente a la de todos los datos.

Si un grupo se aparta mucho, dice algo que el agrupamiento solo no decía.

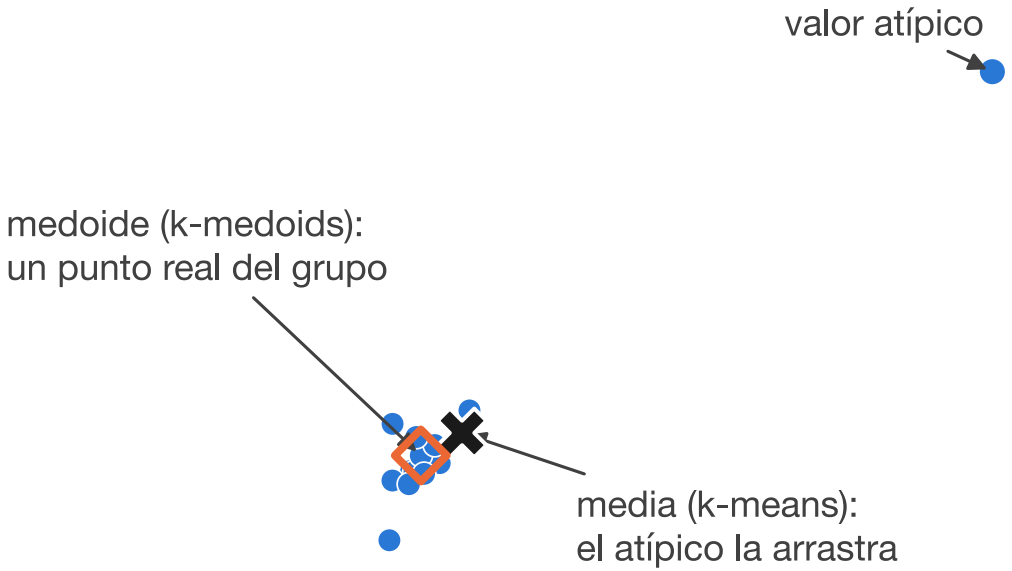
Con la fecha: ¿épocas seguidas o se alternan?

Un grupo que cambia con la semilla o que no se puede explicar no sirve para tomar decisiones.

Limitaciones y alternativas

Cuándo k-means no basta

Sensible a los valores atípicos



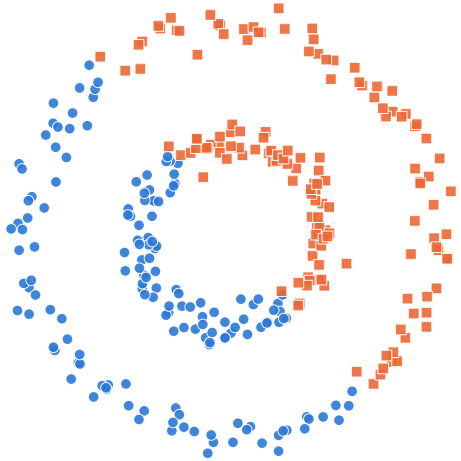
- El centro es una **media**: un punto lejano lo arrastra y puede llegar a quedarse con un grupo para él solo.
- **k-medoids** usa como centro un punto real del grupo, el **medoide**: el que tiene la menor distancia total a los demás. Es más robusto.
- Conviene revisar los atípicos antes de agrupar.

Supone grupos redondos y parecidos

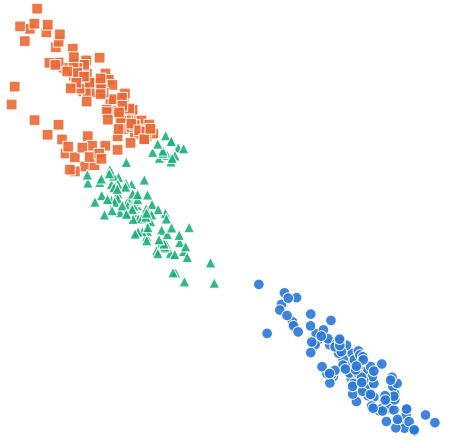
Lunas (k = 2)



Anillos (k = 2)



Grupos alargados (k = 3)



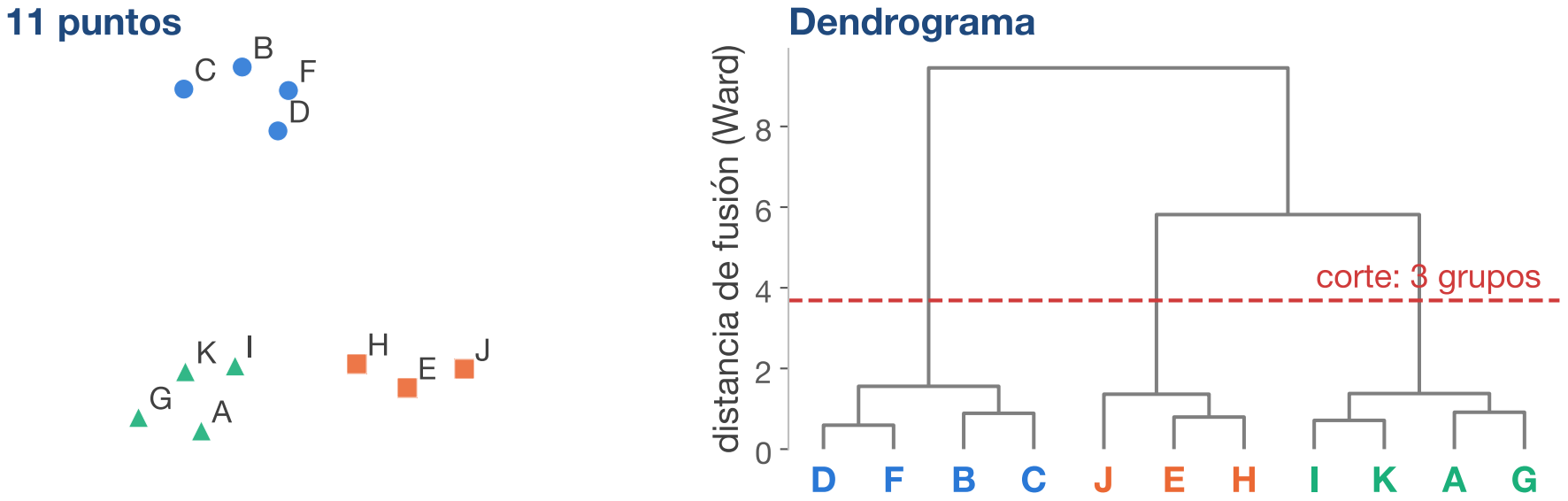
k-means separa los grupos con fronteras rectas a medio camino entre centros. No sigue formas curvas ni alargadas.

Clustering jerárquico aglomerativo

1. Cada punto empieza siendo su propio grupo.
2. Se fusionan los **dos grupos más cercanos**.
3. Se repite hasta que solo queda un grupo: el resultado es un árbol.
4. Se **corta el árbol** a la altura que da los grupos buscados.

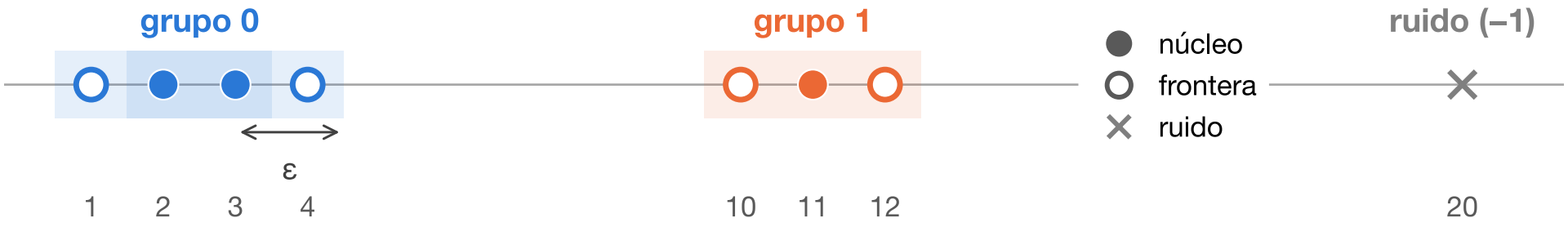
La distancia entre grupos (el *enlace*) cambia el resultado: Ward, completo, medio o simple. El jerárquico **divisivo** hace lo contrario, parte de un solo grupo y lo divide; se usa menos.

El dendrograma



La altura de cada unión es la distancia a la que se fusionan. Un salto grande antes de una fusión sugiere dónde cortar. En Python: `scipy.cluster.hierarchy` o `AgglomerativeClustering`.

DBSCAN paso a paso

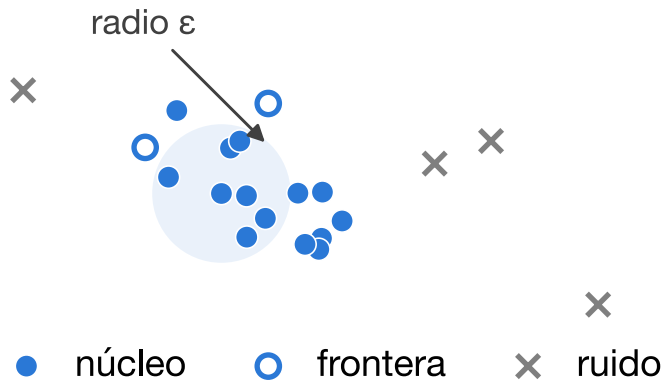


Punto	**1**	**2**	**3**	**4**	**10**	**11**	**12**	**20**
Puntos a ≤ 1,5 (él incluido)	1 2	1 2 3	2 3 4	3 4	10 11	10 11 12	11 12	20
Cuántos son	2	**3**	**3**	2	2	**3**	2	1
Tipo	frontera	**núcleo**	**núcleo**	frontera	frontera	**núcleo**	frontera	ruido
Grupo	0	0	0	0	1	1	1	–1

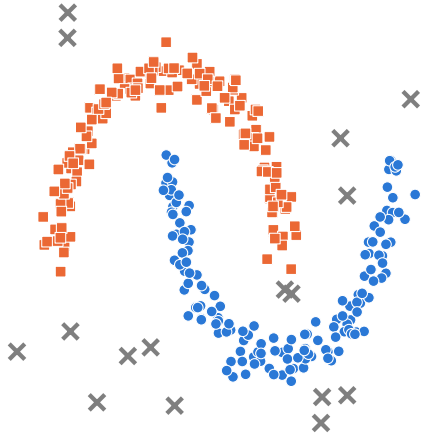
ε = 1,5 y `min_samples` = 3. **Paso 1**: con 3 o más, núcleo; si no, frontera si tiene un núcleo cerca, y ruido si no.
Paso 2: los núcleos cercanos (2 y 3) forman un grupo y cada frontera se une al de su núcleo.

DBSCAN: grupos por densidad

ε = 0,6 y min_samples = 4



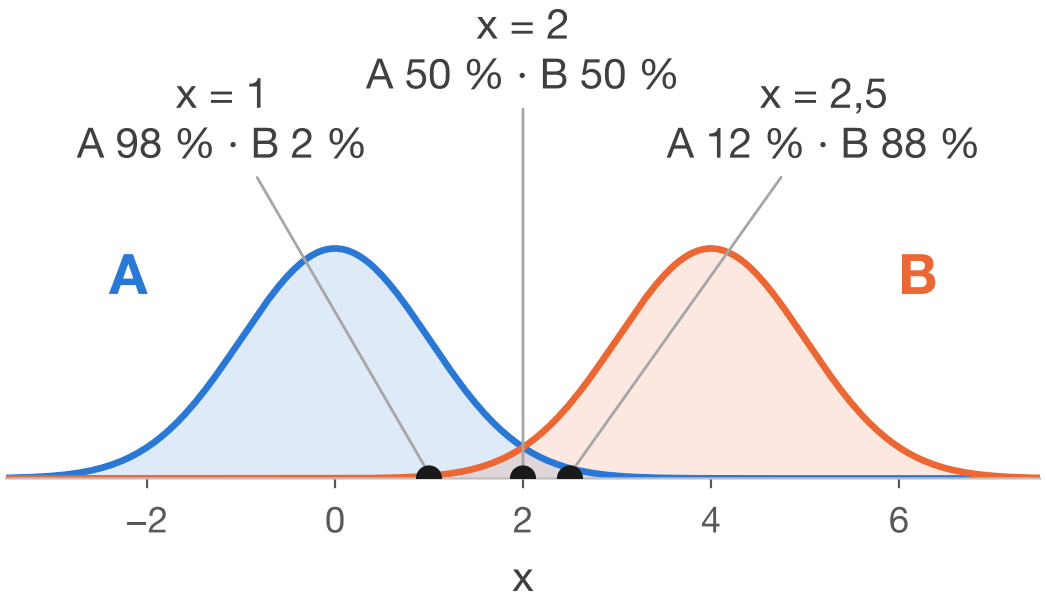
Lunas: 2 grupos y 16 puntos de ruido



Núcleo: al menos `min_samples` puntos a menos de ε. **Frontera**: cerca de un núcleo sin serlo. **Ruido**: ni una cosa ni otra (etiqueta −1). No hay que fijar k y sigue cualquier forma; a cambio, hay que elegir ε.

Mezcla de gaussianas: la idea

- Los datos salen de varias **campanas** (gaussianas) superpuestas. Cada campana es un grupo.
- Cada una tiene **centro**, **forma** (en 2D, una elipse) y **peso**: la fracción de puntos que le toca.
- Cada punto recibe la **probabilidad** de cada campana, según lo alta que esté cada una en ese punto.



k-means pondría x = 2 en un grupo sin avisar de la duda; la mezcla dice «50 %».

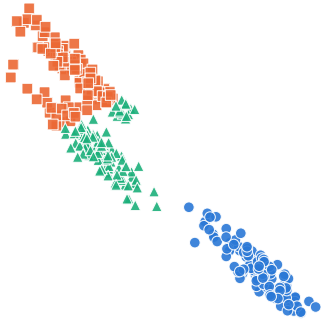
Mezcla de gaussianas: cómo se ajusta

1. **Arrancar** con k campanas (por defecto, a partir de un k-means).
2. **Paso E**: para cada punto, la probabilidad de cada campana. Como «asignar», pero repartiendo.
3. **Paso M**: rehacer cada campana con los puntos pesados por esa probabilidad: centro, forma y peso.
4. Repetir E y M hasta que el ajuste deje de mejorar.

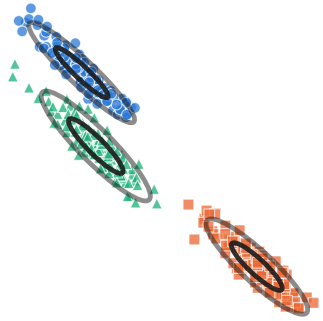
Es el algoritmo **EM** (*expectation–maximization*). Como k-means, depende del arranque: `n_init` lo repite varias veces.

Mezcla de gaussianas frente a k-means

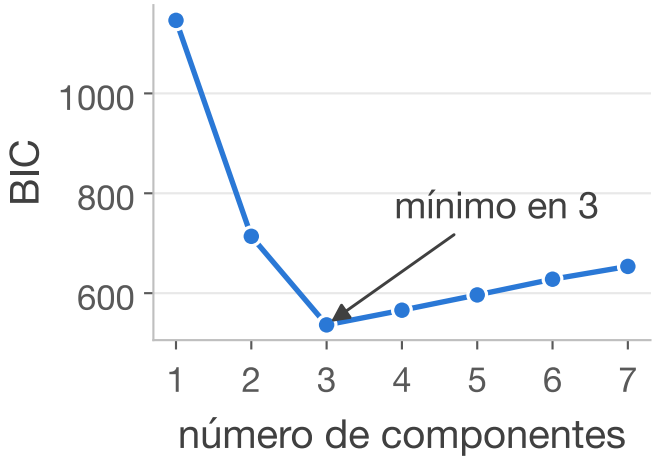
k-means (k = 3)



Mezcla de 3 gaussianas



BIC: elegir el número



Con grupos alargados, la mezcla ajusta una elipse a cada uno. Más campanas siempre ajustan mejor, así que el número se elige con el **BIC**, que penaliza los parámetros: el menor. En scikit-learn, `GaussianMixture` (`predict_proba` da las probabilidades).

Comparación

	k-means	Jerárquico	DBSCAN	Mezcla de gaussianas
Hay que fijar	k	Dónde cortar	ε y `min_samples`	Nº de componentes
Forma de los grupos	Redondos	Según el enlace	Cualquiera	Elipses
Puntos aislados	Los asigna y le afectan	Los asigna	Los marca como ruido	Los asigna
Muchos datos	Rápido	Lento	Intermedio	Intermedio
scikit-learn	`KMeans`	`AgglomerativeClustering`	`DBSCAN`	`GaussianMixture`

Con todos ellos se estandariza antes, y la silueta sirve para comparar agrupamientos.

Agrupar es fácil; decidir cuántos grupos y qué significan, no

Estandariza, prueba varios k, mira el codo, la silueta y su gráfico, y comprueba que los grupos son estables y se pueden explicar.

Práctica: secciones 1 y 2 de `Semana4_ALUMNO.ipynb`.